

The Reproducible Open Coding Kit: A Human and Machine Readable Standard for Coding Qualitative Data

Gjalt-Jorn Peters¹ & Szilvia Zörgő²

This is the second draft of the article "The Reproducible Open Coding Kit: A Human and Machine Readable Standard for Coding Qualitative Data".

This version of the manuscript was rendered by Quarto through the [{zirconia} R package](#). That package is still under development, and as a consequence there may be some glitches. The corresponding git repository is at <https://codeberg.org/sci-ops/ROCK>.

For more information about the ROCK, please see <https://rock.science>. A brief blog post that provides a quick introduction to the standard is available at <https://sciencer.eu/posts/2024-12-rock-basics.html>.

¹ Theory, Methods, and Statistics, Faculty of Behavioural and Health Sciences, Open University of the Netherlands, Valkenburgerstraat 177, Heerlen, the Netherlands; gjalt-jorn.peters@ou.nl & [@matherion@mastodon.nl](https://matherion@mastodon.nl)

² Society Studies, Faculty of Arts and Social Sciences, Maastricht University, Minderbroedersberg 4-6, Maastricht, the Netherlands; s.zorgo@maastrichtuniversity.nl



PREPRINT

The Reproducible Open Coding Kit: A Human and Machine Readable Standard for Coding Qualitative Data

Gjalt-Jorn Peters¹ and Szilvia Zörgő²

¹Theory, Methods, and Statistics, Faculty of Behavioural and Health Sciences, Open University of the Netherlands, Valkenburgerstraat 177, Heerlen, the Netherlands and ²Society Studies, Faculty of Arts and Social Sciences, Maastricht University, Minderbroedersberg 4-6, Maastricht, the Netherlands

Abstract

The Reproducible Open Coding Kit (the ROCK) is a standard for human- and machine-readable qualitative (coded) data. It was designed to be accessible to researchers without dedicated software, while simultaneously offering enough structure to enable programmatic processing. Through this combination and a number of features tailored to these ends, the ROCK lends itself well to facilitate application of open science principles in qualitative research, to catalyse methodological innovation in qualitative research, and to progress towards machine-readability and interoperability of (coded) qualitative data, to ultimately enable truly Findable, Accessible, Interoperable, and Reusable (FAIR) qualitative data. In this canonical reference, we lay out the ROCK standard v1.0. We hope this allows developers to implement the ROCK in existing free and open software, stimulates the development of additional applications, and ultimately, democratizes methodological innovation in qualitative research.

Key words: qualitative methods, open science, interoperability, FAIR data, open data, open materials

Introduction

The world-wide adoption of the United Nations Educational, Scientific and Cultural Organization Recommendation on Open Science is a salient manifestation of the global transition to transparent, accessible science (UNESCO, 2021). Open Science principles and practices include (but are not limited to) preregistration, posting preprints, publishing in Open Access outlets, publishing findable, accessible, interoperable, and reusable (FAIR) open data, publishing open materials (questionnaires, source code, interview schemes, analysis scripts, output, etc), and building community-owned, Open Infrastructure for scientific research (Bilder et al., 2022).

These principles and practices often interact synergistically: for example, Open Infrastructure (i.e., using Free/Libre Open Source Software, FLOSS, tools for study design, data collection and curation, data preparation, analysis, and reporting) often uses Open, non-proprietary file formats, whereas Closed Infrastructure (proprietary software) often uses proprietary file formats. This means that publishing data and materials is considerably less effort and more feasible when using Open Infrastructure than when using Closed Infrastructure.

For quantitative research, extensive Open Infrastructure exists, and these FLOSS applications are steadily replacing conventional Closed Infrastructure alternatives. For example, jamovi (Navarro and Foxcroft, 2025; ŞAHİN and AYBEK, 2020), JASP (Love et al., 2019), R (R Core Team, 2025) and Python (Van Rossum and Drake, 2009) are four examples of popular software packages that facilitate the sharing of data, analyses, and output. This sprawling landscape of FLOSS tools is strengthened by a variety of open file formats that facilitate data exchange, such as the comma-separated values format (.csv), OpenDocument Spreadsheet (.ods), Office Open XML (.xlsx), the JavaScript Object Notation format (JSON; .json), and the JSON superset YAML [a recursive acronym for YAML Ain't Markup Language; Ben-Kiki et al. (2009)].

However, for qualitative research, the most popular tools remain Closed Infrastructure [i.e., proprietary, non-Open Science software; see e.g. Evers (2018)]. In 2016, the companies producing eight popular proprietary qualitative data analysis tools together formed the Rotterdam Exchange Format Initiative (REFI) and proceeded to develop two Extensible Markup Language (XML)-based open standards for qualitative research: the REFI-QDA Project and the REFI-QDA Codebook standards (Evers et al., 2020; Rotterdam Exchange Format Initiative, 2019). The REFI-QDA Project is an extensive and complex format, capable of storing data, the codebook, and the coded fragments. REFI-QDA Codebook is much simpler, only capable of storing a codebook without data and coded data fragments.

Despite its complexity, the REFI-QDA Project format does not support all features of the proprietary software packages developed by the eight companies, nor does it allow what the REFI calls “round-tripping”: exporting a project from one software package to another, and then importing it back into the first software package. In addition, while the REFI-QDA Codebook format is implemented in a number of FLOSS applications, as yet the REFI-QDA Project standard has been implemented in only one FLOSS package: QualCoder (Brailas et al., 2023; Rotterdam Exchange Format Initiative, 2019). The format’s complexity may

be one of the reasons FLOSS tools are slow to implement this standard: because FLOSS is generally free, there are often fewer developers available, forcing strict prioritization of features.

In addition to the challenges that come with implementing such a complex format, the fact that the REFI-QDA standards are XML-based also means that viewing REFI-QDA files requires dedicated software, be it Open or Closed. Although XML is well suited for data serialization, it is hard to read for humans. As such, the REFI-QDA standards can be seen as the equivalent of Office Open XML or OpenDocument Spreadsheet for qualitative data: a powerful, versatile standard that is machine-readable but not human-readable. This analogy highlights that although for quantitative data, human-readable open formats exist (such as CSV and YAML), there is a need for such formats for qualitative data.

In the present contribution, we introduce the Reproducible Open Coding Kit (the ROCK), an open standard for qualitative data that was designed to be both machine- and human-readable. Like files in the Markdown and YAML formats, coded qualitative data in the ROCK format can be directly read by humans without requiring specialized software. In addition, because the ROCK standard is relatively rudimentary it is also relatively easy to implement in existing software. It is our hope it will contribute to an expanding landscape of Open Infrastructure solutions for qualitative research. We will first provide the technical description of the ROCK standard. We will then discuss how it can facilitate Open Science and provide guidance for using the ROCK as a qualitative researcher, including a brief description of existing implementations, and for implementing the ROCK as a software developer.

The Reproducible Open Coding Kit 1.0

In designing the Reproducible Open Coding Kit (the ROCK), our decisions were guided by the desire to retain directly human-readable files, similar to the vision underlying the Markdown (Gruber and Swartz, 2004) and YAML standards (Ben-Kiki et al., 2009). ROCK files are always plain-text files, although we suggest using “.rock” as file extension rather than “.txt”. For textual and transcribed audio data, this file format is the most widely supported format. However, this also means that for all other types of data, a representation in plain text format is first required. This can be accomplished in a number of ways. For example, for video or audio data, timestamps can be used to encode alignment of the plain text file to the original stream. The plain text file can then be coded, an example of which will be shown later.

Because the ROCK standard contains many intertwined concepts, in the following description it is inevitable that in the descriptions of ROCK concepts, we use terms that are only later fully defined. These terms are printed in *italic*, and can be found in the glossary.

Examples of data sources coded with the ROCK

A minimally coded example of a ROCK *source* is provided in Figure 1, with line numbers added before each line. This example only contains the transcript and several added codes (between double square brackets). The transcript is a combination of three fragments from an interview for YouTube channel Soft White Underbelly by Mark Laita (see <https://youtu.be/W02HJ8j3vAk>). Specifically, the first nine lines came from <https://youtu.be/W02HJ8j3vAk?t=461>; lines 10-14 from <https://youtu.be/W02HJ8j3vAk?t=692>; and lines 15-29 from <https://youtu.be/W02HJ8j3vAk?t=816>.

Figure 1: A transcript with several codes in the ROCK standard, where each sentence is on a separate line. Line numbers have been added before every line; although the text is wrapped, this enables identification of separate lines. In the ROCK standard, lines are called *utterances*.

```
01 Mark: Do you have any dreams or goals or anything you'd like to see happen in
    your future? You're still very young.
02 I want to just be happy, bro.
03 The only dream I probably got is just to be happy.
04 'Cause just because I smile every day, just because I walk around like I'm cool,
    I'm not happy. [[dissonance]]
05 (mumbles) look at myself in the mirror without feeling like, "Damn, this is what
    life is." [[dissonance]]
06 Mark: You get depressed a lot?
07 Yeah, but I don't show it. [[dissonance]]
08 Because if you show it, then it's a sign of weakness and when you're weak is
    when the people prey on you most. [[abuse>A.physical]] [[dissonance]]
09 I just learned how to make sure that everyone thinks I'm cool. [[dissonance]]
10 You know, these streets will take something beautiful and they'll just like crush it
    and just like make sure that that's nothing (mumbles). [[abuse>A.physical]] [[loss]]
11 Because out here there's just one thing.
12 Money talks, bullshit walks. [[abuse>A.physical]]
13 You could pay somebody to do anything out here, bro, and they'll do it.
14 If it's the price is right.
15 So everything I ever thought was right, isn't really right out here,
    it's wrong. [[loss]]
16 Whatever I thought was wrong is right. [[loss]]
17 What is, isn't, what isn't is. [[loss]]
18 I seen children, old people, get killed just 'cause they don't want to comply
    with what the fuck these people want. [[abuse>A.physical]] [[abuse>A.emotional]]
19 I've been raped so many times I can't even count on
    one hand. [[abuse>A.physical]] [[abuse>A.emotional]]
```

```

20 My grandmother died I couldn't go to her funeral. [[loss]]
21 My father died I couldn't go to his funeral. [[loss]]
22 And yet I still find a reason to smile every day.
23 I've been kidnapped before at gunpoint. [[abuse>A.physical]] [[abuse>A.emotional]]
24 I've been robbed. [[abuse>A.physical]] [[loss]]
25 I've been stabbed, all that shit. [[abuse>A.physical]]
26 Sucks, 'cause nobody will ever know what a person like me goes through
   on a daily basis or give a fuck really, honestly. [[dissonance]]
27 I go to the store I see people that have a life, they have somewhere to go,
   have somewhere to be.
28 They look at me with disgust and I just think to myself like, "Damn, I wonder
   how they would look at me if they knew what I've been through." [[dissonance]]
29 And the only reason I'm doing this because I hope that maybe somebody who
   gives a fuck will listen.

```

Figure 1 shows that lines 4, 5, 7-10, 12, 15-21, 23-26, and 28 have been coded: these lines contain code identifiers delimited with pairs of square brackets. Most coding instances represent a single code, but on lines 8, 10, 12, 18, 19, and 23-25, the greater-than symbol is used to represent the parent-child relationship between two codes. Lines 10, 18, 23, and 24 contain multiple separate codes. This transcript can be read without any dedicated software; if a reader knows they are reading coded data, even the convention that the codes are between square brackets can likely be inferred. In addition to being human-readable as-is, the convention used by these codes allows software to import the codes and show the code structure, show all fragments coded with a given code, or generate visualizations. Two more extensive examples are provided in Figures 2 and 3.

Figure 2: The same transcript as shown in Figure 1, but with so-called *utterance* identifiers added before every line, a class instance identifier, section breaks, and anchors.

```

--+-{ 2022-10-17 10:07:32.752 CEST (UTC+2) }-+--
[[caseId=kristina]]
01 [[uid=7lm5s187]] Mark: Do you have any dreams or goals or anything you'd like to
   see happen in your future? You're still very young.
02 [[uid=7lm5s188]] I want to just be happy, bro.
03 [[uid=7lm5s189]] The only dream I probably got is just to be happy.
04 [[uid=7lm5s18b]] 'Cause just because I smile every day, just because I walk around
   like I'm cool, I'm not happy. [[dissonance]]
05 [[uid=7lm5s18c]] (mumbles) look at myself in the mirror without feeling like, "Damn,
   this is what life is." [[dissonance]]
06 [[uid=7lm5s18d]] Mark: You get depressed a lot?
07 [[uid=7lm5s18f]] Yeah, but I don't show it. [[dissonance]]
08 [[uid=7lm5s18g]] Because if you show it, then it's a sign of weakness and when you're
   weak is when the people prey on you most. [[abuse>A.physical]]
09 [[uid=7lm5s18h]] I just learned how to make sure that everyone thinks I'm cool.
   {--<<pause>>---
10 [[uid=7lm5s18j]] You know, these streets will take something beautiful and they'll
   just like crush it and just like make sure that that's nothing
   (mumbles). [[abuse>A.physical]]
11 [[uid=7lm5s18k]] Because out here there's just one thing.
12 [[uid=7lm5s18l]] Money talks, bullshit walks. [[abuse>A.physical]]
13 [[uid=7lm5s18m]] You could pay somebody to do anything out here, bro, and they'll do it.
14 [[uid=7lm5s18n]] If it's the price is right.
   {--<<pause>>---
15 [[uid=7lm5s18p]] So everything I ever thought was right, isn't really right out here,
   it's wrong.
16 [[uid=7lm5s18q]] Whatever I thought was wrong is right.
17 [[uid=7lm5s18r]] What is, isn't, what isn't is.
18 [[uid=7lm5s18s]] I seen children, old people, get killed just 'cause they don't want
   to comply with what the fuck these people want. [[abuse>A.physical]]
   [[abuse>A.emotional]]
19 [[uid=7lm5s18t]] I've been raped so many times I can't even count
   on one hand. [[abuse>A.physical]] [[abuse>A.emotional]]
--+-{ 2022-10-17 10:13:06.239 CEST (UTC+2) }-+--
20 [[uid=7lm5s18w]] My grandmother died I couldn't go to her funeral.
21 [[uid=7lm5s18x]] My father died I couldn't go to his funeral.
22 [[uid=7lm5s18y]] And yet I still find a reason to smile every day.
23 [[uid=7lm5s18z]] I've been kidnapped before
   at gunpoint. [[abuse>A.physical]] [[abuse>A.emotional]]
24 [[uid=7lm5s190]] I've been robbed. [[abuse>A.physical]]
25 [[uid=7lm5s191]] I've been stabbed, all that shit. [[abuse>A.physical]]

```

```

26 [[uid=71m5s192]] Sucks, 'cause nobody will ever know what a person like me goes through
    on a daily basis or give a fuck really, honestly.
27 [[uid=71m5s193]] I go to the store I see people that have a life, they have somewhere
    to go, have somewhere to be.
28 [[uid=71m5s194]] They look at me with disgust and I just think to myself like, "Damn, I
    wonder how they would look at me if they knew what I've been through."
29 [[uid=71m5s195]] And the only reason I'm doing this because I hope that maybe somebody
    who gives a fuck will listen.
--+-{ 2022-10-17 10:18:52.682 CEST (UTC+2) }-+--

```

In this second example, every line is preceded by a so-called “utterance identifier” (UID). These uniquely identify each line, allowing unequivocal reference to *data fragments* as well as merging codes from multiple versions of the same *source* in the case of multiple independent coders. In addition, a *class instance identifier* has been added on the second line, indicating that all following utterances (i.e. all following lines; see section “Sources and Utterances” below) belong to case “kristina” (“caseId” stands for “case identifier”, where “case” is used as a synonym for “participant”). This enables efficient association of attributes (such as the participant’s age) to those utterances linked to that participant. The data have been segmented with two section breaks identified by “pause”, and three anchors have been added (the lines with the date and time), which allow synchronizing codes from multiple data streams. Where in Figure 2, utterances were defined as sentences (and so, every sentence was on a separate line; see section “Utterances”), another common definition is in number of characters (also see section “Sources and Utterances” below). Figure 3 shows the same example, but with utterances defined as 40 characters (and so, lines consist of text fragments of 40 characters or less).

Figure 3: The same transcript as shown in Figure 1, but with *utterances* defined as sequences of 40 characters (while respecting word boundaries, so effectively word wrapping the source at 40 characters). The original line numbers have been included as reference; note that because in this version of the transcript, sentence endings (e.g. periods) do not end lines, these original line numbers do not always coincide with new lines in this version.

```

01 [[uid=7ztxtgz]] Mark: Do you have any dreams or goals
    [[uid=7ztxtgzx]] or anything you'd like to see happen in
    [[uid=7ztxtgzy]] your future? You're still very young.
02 [[uid=7ztxtgzz]] I want to just be happy, bro. The only
03 [[uid=7ztxth00]] dream I probably got is just to be
04 [[uid=7ztxth01]] happy. 'Cause just because I smile
    [[uid=7ztxth02]] every day, just because I walk around    [[dissonance]]
05 [[uid=7ztxth03]] like I'm cool, I'm not happy. (mumbles)
    [[uid=7ztxth04]] look at myself in the mirror without
    [[uid=7ztxth05]] feeling like, "Damn, this is what life    [[dissonance]]
    [[uid=7ztxth06]] is."
06 [[uid=7ztxth07]] Mark: You get depressed a lot?
07 [[uid=7ztxth08]] Yeah, but I don't show it.                [[dissonance]]
08 [[uid=7ztxth09]] Because if you show it, then it's a
    [[uid=7ztxth0b]] sign of weakness and when you're weak
09 [[uid=7ztxth0c]] is when the people prey on you most. I    [[abuse>A_physical]] [[dissonance]]
    [[uid=7ztxth0d]] just learned how to make sure that
10 [[uid=7ztxth0f]] everyone thinks I'm cool. You know,        [[dissonance]]
    [[uid=7ztxth0g]] these streets will take something
    [[uid=7ztxth0h]] beautiful and they'll just like crush    [[abuse>A_physical]]
    [[uid=7ztxth0j]] it and just like make sure that that's    [[loss]]
11 [[uid=7ztxth0k]] nothing (mumbles). Because out here
12 [[uid=7ztxth0l]] there's just one thing. Money talks,
    [[uid=7ztxth0m]] bullshit walks.                            [[abuse>A_physical]]
13 [[uid=7ztxth0n]] You could pay somebody to do anything
14 [[uid=7ztxth0p]] out here, bro, and they'll do it. If
15 [[uid=7ztxth0q]] it's the price is right. So everything
    [[uid=7ztxth0r]] I ever thought was right, isn't really    [[loss]]
16 [[uid=7ztxth0s]] right out here, it's wrong. Whatever I
17 [[uid=7ztxth0t]] thought was wrong is right. What is,        [[loss]]
18 [[uid=7ztxth0w]] isn't, what isn't is. I seen children,      [[loss]]
    [[uid=7ztxth0x]] old people, get killed just 'cause they
    [[uid=7ztxth0y]] don't want to comply with what the fuck    [[abuse>A_physical]] [[abuse>A_emotional]]
19 [[uid=7ztxth0z]] these people want. I've been raped so
    [[uid=7ztxth10]] many times I can't even count on one      [[abuse>A_physical]] [[abuse>A_emotional]]
20 [[uid=7ztxth11]] hand. My grandmother died I couldn't go    [[loss]]
21 [[uid=7ztxth12]] to her funeral. My father died I
22 [[uid=7ztxth13]] couldn't go to his funeral. And yet I        [[loss]]
    [[uid=7ztxth14]] still find a reason to smile every day.
23 [[uid=7ztxth15]] I've been kidnapped before at gunpoint.    [[abuse>A_physical]] [[abuse>A_emotional]]

```

```

24 [[uid=7ztxth16]] I've been robbed. I've been stabbed,      [[abuse>A_physical]] [[loss]]
26 [[uid=7ztxth17]] all that shit. Sucks, 'cause nobody      [[abuse>A_physical]]
  [[uid=7ztxth18]] will ever know what a person like me
  [[uid=7ztxth19]] goes through on a daily basis or give a
27 [[uid=7ztxth1b]] fuck really, honestly. I go to the        [[dissonance]]
  [[uid=7ztxth1c]] store I see people that have a life,
  [[uid=7ztxth1d]] they have somewhere to go, have
28 [[uid=7ztxth1f]] somewhere to be. They look at me with
  [[uid=7ztxth1g]] disgust and I just think to myself
  [[uid=7ztxth1h]] like, "Damn, I wonder how they would
  [[uid=7ztxth1j]] look at me if they knew what I've been    [[dissonance]]
29 [[uid=7ztxth1k]] through." And the only reason I'm doing  [[dissonance]]
  [[uid=7ztxth1l]] this because I hope that maybe somebody
  [[uid=7ztxth1m]] who gives a fuck will listen.             [[intensity||moderate]]

```

The elements of the ROCK standard

We will now discuss each element defined in the ROCK in detail. We will refer to the line numbers from these examples where appropriate. Because these descriptions inevitably involve ROCK terms and concepts that will only be explained later, these have been included in the glossary. These terms are printed in *italic*.

Sources and utterances

The ROCK term for a plain text file is a *source* (like the REFI term; also see the “REFI terms and ROCK terms” section below). Sources consist of one or more data fragments called *utterances*, and *codes* are typically applied at the utterance level (but see section “Character-level coding” below). How a source is segmented into utterances can be determined by what is convenient for coding or by what is considered a meaningful codable unit such as sentences (sequences of words separated by periods, question or exclamation marks, or ellipses) or paragraphs; sequences of words of at most 40 characters; or data fragments collected in the same 10-second span. Utterances are separated by line breaks (i.e., ASCII character 10, unicode U+000A). Typically, ROCK sources are plain text files with the “.rock” extension, but software working with ROCK files ideally also accepts “.txt” extensions (or even other extensions).

Identifiers

In the ROCK, identifiers play an important role as they represent coding, segmentation, and specification of metadata. These metadata, in turn, are used as the scaffolding to attach *attributes* to coded *utterances*. Finally, identifiers are used to uniquely identify data fragments, enabling unequivocal reference as well as merging multiple versions of a coded source (e.g. when working with multiple coders or multiple coding sessions). Identifiers are unique character strings that can only consist of lowercase and uppercase Latin letters (i.e., “a” to “z” and “A” to “Z”), Arabic numerals (i.e., “0” to “9”), and underscores (i.e., “_”), and must always start with a letter¹. In the ROCK, identifiers are used to encode most things that are not raw data. Identifiers can identify *utterances*, *codes*, *section breaks*, and *class instances* (these will be explained below, but see the glossary for quick reference). These identifiers are delimited with double square brackets (i.e., “[” on the left side and “]” on the right side), except for section break identifiers, which are delimited with three hyphens and two less-than and two greater-than symbols (i.e., “---<” on the left side and “>---” on the right side).

Codes

In the ROCK, when a data fragment is coded with a code, that represents the decision by a qualitative researcher that the concept (e.g., a phenomenon, construct, process, or symbol) represented by that code is expressed in, implied, or otherwise manifest in that data fragment (or to part of the fragment; see section “Character-level coding” below). For this purpose, one or more code identifiers are appended to an utterance (i.e. a data fragment). The ROCK 1.0 supports three organizational modes for codes: flat coding, hierarchical coding (or tree coding), and network coding. The simplest is flat coding, where no relationship between codes is encoded. When using flat coding, code identifiers are directly placed between their delimiters, for example “[[codeId]]”. With hierarchical coding or nested coding, parent-child relationships are indicated using the hierarchy marker, a greater-than symbol (“>”). For example, to specify a child code, you can use “[[codeId>childCodeId]]”. It is possible to chain together an arbitrary number of code identifiers in this way, for example “[[parentCodeId>childCodeId>grandchildCodeId]]”.

Finally, when using network coding (Peters et al., 2022b), three identifiers have to be specified: the code identifier for the code representing the originating node, the code identifier for the code representing the receiving node, and a relationship type. The first two are separated by a hyphen (-) and a greater-than sign (>), and the relationship type is separated by two pipes (||), for example “[[originatingCode->receivingCode||relationshipType]]” (there are no prespecified relationship types: like code identifiers, these can be arbitrary character strings provided they satisfy the identifier naming conventions). As yet, the ROCK does not include a notation enabling hypergraph support.

The entire sequence of characters that together specifies the code or codes to apply is called a *coding* or *coding pattern*. A coding, therefore, always refers to the attachment of a code (or multiple codes, when using tree coding or network coding) to a data fragment. Multiple codings can be applied to a given utterance. Codings on lines that contain no data (i.e. only whitespace and/or other codings) are considered to apply to the closest preceding utterance. In other words, it does not matter whether a code is applied on the same line, the following line, or even after one or more empty lines.

¹ There is one exception to this rule: *class instance identifiers* can start with, or even only consist of, a digit.

Code values

It is also possible to specify a value for a flat code, for example when using magnitude coding. For example, when participants' ages are registered during the interview, or when a coder wants to indicate the intensity of a certain expression, code values can be used for this. Code values consist of a code identifier followed by double pipes (||) and the value, for example "[[age||43]]" or "[[intensity||moderate]]". This last example is included in Figure 3 at UID "7ztxth1m".

Character-level coding

By default, codes appended to an utterance apply to the entire utterance. This decision was made to optimize the legibility of coded data (by humans). In many cases, meaning does not exist in one or two words but also in the context created by what preceded, and attaching a code often means that in the data, a coder observed that which the code represents, which implicitly includes the surrounding context. In such cases, sub-utterance coding has little added value. However, sometimes it is desirable to be able to indicate more specifically where a code applies. This is supported by the ROCK by inserting, between the code identifiers and the closing delimiters, a semicolon (;), followed by a number, followed by a dash (-), followed by a number again. These numbers indicate the first coded character in the utterance and the last coded character (inclusive), respectively. For example, "[[intensity||low;12-20]]" would code the 12th to the 20th character with low intensity; and "[[advantages>specificity;24-42]]" would code characters 24-42 with the tree code with identifier 'specificity', nested under a parent code identified by 'advantages'. Omitting the first number is equivalent to specifying 1 (i.e. the code starts at the first character), and omitting the second number is equivalent to specifying the number of the last character on the line. This means that appending ";-" to a coding is equivalent to not appending anything at all; the code(s) will apply to the entire utterance. As such, appending "[[a_code;1-20]]" is equivalent to appending "[[a_code;-20]]", and for a line that has 67 characters, appending "[[a_code;20-67]]" is equivalent to appending "[[a_code;20-]]".

Utterance identifiers

Utterance identifiers (UIDs) are unique identifiers with the identifier label "uid" (very similar to a *class identifier*), separated from the identifier by an equals sign (=) or a colon (:), for example "[[uid=7kbkkc71]]" or "[[uid:7kbkkc71]]". Designating utterance identifiers (UIDs) to every codable data fragment (utterance) has a number of benefits. First, they allow unequivocal reference to a given data fragment (for example in reports or when discussing the data). Second, they make it possible to merge multiple copies of the same source, coded by different coders, into one primary source. Third, their unequivocal reference to specific utterances enables programmatic coding. Fourth, combined with the *timestamped source identifier* (TSSID; see below), and project (Short)DOI, they provide globally unique identifiers to a given data fragment (see the TSSID section).

It is strongly recommended to use SQUIDS [Short Quasi Unique Identifiers; Peters (2025)] as utterance identifiers. SQUIDS are 8-character strings that express a Unix timestamp (i.e., the number of non-leap seconds that have elapsed since 00:00:00 UTC on 1 January 1970), multiplied by 100 to obtain centisecond precision, rounded, and converted to a base 30 system that consists of the ten Arabic digits followed by the Latin alphabet but omitting a, e, i, o, u, and v. This system has two benefits. First, it enables converting a SQUID back to the original timestamp. For UIDs, that generally means the moment the UIDs were attached to the data fragments. Second, in a consecutive series of SQUIDS, the last character only recurs every 30 SQUIDS; a given sequence of two last characters only recurs every 900 SQUIDS; a given sequence of three last characters recurs only every 27 000 SQUIDS; and a given sequence of four last characters only recurs once every 810 000 SQUIDS. As a consequence, within one transcript, the last three characters of a UID are guaranteed to suffice as unique references to an utterance, and within a project, the last four characters of every UID are typically unique. This enables very efficient unequivocal reference to specific data fragments (see the Nesting section for an example).

Segmentation

Sources can be divided into sections using section breaks, providing means for segmentation using multiple parallel segmentation schemes, where each unique section break identifier represents an independent segmentation scheme. For example, if a source contains both "---<<interview_topic>>--" and "---<<turn_of_talk>>--", every occurrence of the "interview_topic" section break signifies segmentation into interview topics, and every occurrence of "turn_of_talk" signifies segmentation into turns of talk, where many turns of talk can occur per interview topic (without the need for clean nesting). Other examples of segmentation criteria are protracted pauses in speaking or elapsed time intervals.

Nesting

In addition to segmentation, the ROCK also supports nesting of utterances within other utterances. This is useful for representing threaded data, such as sets of replies as can be found on social media (e.g., Mastodon). Nesting is indicated by one or more tildes ("~"), where the tilde has to be the first non-data character on a line. In other words, it can follow whitespace, UIDs, or other codings, but should precede any data. Each further nesting level is indicated by an additional tilde. To indicate that an utterance (i.e. a data fragment) is nested within the preceding utterance (as is the case when it is a reply to it), that utterance is preceded by a single tilde ("~"); and if the utterance following it is a reply to that reply, that is preceded by two tildes directly following each other ("~~"). If the fourth utterance is a reply to the first utterance again, it is again preceded by one tilde. Inspect the following example:

```
[[sourceDomain=mastodon.social]]
[[uid=7lm5s187]] Somebody's initial post, possibly
[[uid=7lm5s188]] spread over two utterances.
[[uid=7lm5s189]] ~ The first reply to that post
[[uid=7lm5s19b]] ~~ A reply to that first reply.
[[uid=7lm5s19c]] ~~ Either a continuation or a new reply at the same level
```

```
[[uid=7lm5s19d]] ~ The second reply to the initial post
[[uid=7lm5s19f]] ~ Again, either a continuation or a third reply
[[uid=7lm5s19g]] Another post, not a part of the previous thread
```

As is illustrated here, tildes indicate which utterance is an utterance's "parent" in a situation where utterances are nested; but although it can seem as if they do, they do not group utterances together. To encode such metadata, class instance identifiers can be used. This may look, for example, like this:

```
[[sourceDomain=mastodon.social]]
[[uid=7lm5s187]] Somebody's initial post, possibly                [[postId=1]]
[[uid=7lm5s188]] spread over two utterances.
[[uid=7lm5s189]] ~ The first reply to that post                  [[postId=2]]
[[uid=7lm5s19b]] ~ A reply to that first reply.                  [[postId=3]]
[[uid=7lm5s19c]] ~ Either a continuation or a new reply at the same level
[[uid=7lm5s19d]] ~ The second reply to the initial post          [[postId=1]]
[[uid=7lm5s19f]] ~ Again, either a continuation or a third reply [[postId=3]]
[[uid=7lm5s19g]] Another post, not a part of the previous thread [[postId=4]]
```

The above clarifies that the utterances with the last UID characters 87 and 88 belong to the same poster (poster "1"); as do utterances 9b and 9c (poster "3"); but utterances 9d and 9f belong to different posters (posters "1" and "3", respectively).

Classes and class instances

Class instances are used to encode additional information about data, often about the data's provenance. To facilitate this, they can be used to efficiently attach specific attributes to utterances (see next section). A common class is what REFI terms a "case", often used to represent data providers (e.g., participants). In the ROCK, classes are more abstract, simply representing means to encode data provenance or information relating to processing or analysis of data (optionally associating attributes to utterances). For example, in the previous section, class instances were used to register the domain of the social media service where a thread originated (`sourceDomain=mastodon.social`) as well as the posters contributing to the thread (e.g., `postId=1`). Similarly, for interviews, classes can be used to encode characteristics of the data collection such as attributes of the location (with a class identifier such as "location" and class instance identifiers such as "library" or "university") or the time of day (with a class identifier such as "time" and class instance identifiers such as "morning", "afternoon", and "evening"), as well as the transcriber (with a class identifier such as "transcriber" and class instance identifiers such as "Alex" or "Bobby") or the coder (with class identifiers such as "coderId" and class instance identifiers such as "Charlie" or "Dylan").

Class instance identifiers are persistent identifiers: they apply not only to the utterance where they appear, but also to all consecutive utterances until the source ends or another instance identifier of the same class is encountered. They are specified using the class identifier followed by a colon (:) or equals sign (=) and the class instance identifier, for example "[[caseId=1]]", "[[location:library]]", "[[time:evening]]", "[[transcriber=Alex]]", or "[[coderId=Dylan]]".

For example, if a source represents a transcript from a focus group with three participants identified by "p1", "p2", and "p3", the contributions of each can be marked with "[[caseId=p1]]", "[[caseId=p2]]", and "[[caseId=p3]]", respectively. If the focus group took place in the morning in a library, this could be indicated by adding "[[location=library]]" and "[[time=morning]]" to the beginning of the source. This will allow the specification of *attributes* for these class instances, such as the ages of "p1", "p2", and "p3", or characteristics of the data collection locations or the coders (e.g. their discipline, or specific categorized elements from their positionality). This enables collecting the data fragments coded with a given code from those participants of a certain age (e.g. "thirties"); present in data from a location with certain properties (e.g. "inside"), or applied by coders from a given background (e.g. "anthropology").

For specific use cases, class instance identifiers can also be used to organize the data in other ways. For example, when coding transcripts for cognitive interviews, class instance identifiers can be used to indicate which data are about which items (e.g. "[[uid=item.4]]"; "uid" stands for Unique Item Identifier) or probes (e.g., "[[prbid=item.4]]"; prbid stands for Probe Identifier), and when using data streams from multiple origins, class instance identifiers can be used to encode these (e.g. "[[origin.site=wikipedia]]"), then enabling efficient attribute attachment based on these identifiers. Finally, it is possible to encode states with class instance identifiers, which allow the analysis of state transitions, for example "[[state=problem.solving]]" (for an example, see Zörgö et al., 2025).

Anchors

Anchors are lines that start with "---{", that end with "}----", and that in between those two delimiters contain a sequence of characters that marks the specific location in the data source (e.g., "2022-10-17 10:18:52.682 CEST (UTC+2)"). Anchors enable calibration of multiple data streams, making it possible to map codes from one or multiple streams onto one primary stream. This in turn enables coding data from various data streams independently and combining the results afterwards. Since anchors will typically represent time, the recommendation is to use the ISO 8601 standard as in the examples above. Because anchors do not have to satisfy the requirements for identifiers, more human-readable versions of the ISO 8601 standard can be used, again as illustrated in these examples. Note that using times and dates are a recommendation but not a requirement: anchors can have mostly free contents (specifically, they have to satisfy regular expression "[a-zA-Z0-9-:.,+@!#%&*)(\|><|~=-]"). To enable calibration, therefore, the exact same anchors must be used across data streams (i.e. concurrent sets of sources).

Comments

Lines that start with one or more hash or pound signs ("#") can be used for comments. ROCK parsers should ignore these lines.

Notes

Notes can be embedded using opening delimiter “--{” and closing delimiter “}--”. Notes are attached to the most proximal preceding data fragment. Note can span multiple lines and can also contain keyed values. For keyed values, the key (field name or variable name) immediately follows the opening delimiter, and is separated from the stored value by “={”. Notes can be used to attach more complicated information to data fragments than is possible with codes. Keyed values can be used to encode information about that data fragment (e.g. when storing information about observations), about the note (e.g. when documenting information about an observation session), or both. The following example illustrates how notes can be specified in the ROCK.

```
This is a data fragment.
--{ This is a brief note attached to it. }--
--{ It has three lines, each delimited   }--
--{ with the note delimiter.             }--
--{ exampleKey }={ the keyed value      }--

This is a another
data fragment.

--{

This is second note that is attached to the
second line of that second data fragment.

}--
```

All consecutive lines containing note information are considered one note. Empty lines start new notes, unless they are within note delimiters (all contents between note delimiters are considered part of the note, unless they start with a hash in which case they are considered comments and ignored). This means an arbitrary number of notes can be attached to a data fragment. Keyed values are always associated to the note they are connected to. As such, if two key-value pairs are separated by an empty line, they should be imported as separate notes.

Time stamped source identifiers

The class identifier “tssid” is reserved as the default identifier for time stamped source identifiers (TSSIDs). A TSSID is a date and time in ISO 8601 format in the Coordinated Universal Time (UTC) time zone and omitting seconds. This yields, for example, “20250809T0900Z”. The recommendation is to use the time when the data collection of the data in that source started. When appending an underscore and UID to this pattern (yielding e.g. “20250809T0900Z_80p2tf4t”), the resulting pattern is very likely to be universally unique. Combined with a (Short)DOI (for example, that of a projects’ repository or of a publication about the project that includes the data), this allows worldwide unequivocal reference to data fragments (e.g., across publications).

An example is the identifier “p2bp_20230404T1600Z_7qzb6kw9”. This can be reconstructed into three elements. The first is shortDOI (p2bp) which, through <https://shortdoi.org>, can be converted to DOI “10.2196/64901”. Visiting <https://doi.org/10.2196/64901> leads to the Open Science Framework repository at <https://osf.io/ynt7a>, which links to the Git repository at <https://gitlab.com/sos-tips/main>. When those data are downloaded, searching the included files for the second element, the TSSID (20230404T1600Z), yields a number of files, each representing the data throughout different stages of the analysis as specified in the directory containing the file and the filename. Finally, searching for the last element, the UID (7qzb6kw9), in these files yields data fragment “C: So a unique number associated with an article or sometimes it, you actually get them for a lot of things, now datasets, softwares, a unique identifier that can be cited. It doesn’t mean by itself that it’s peer reviewed, but peer reviewed articles will have a DOI, at least all the recent ones.”. In this project, two codings have been applied to that data fragment (“[[APPR>TA.DOI.pres]]” and “[APPR>TA.SciMet.pres]]”).

Generally, as in the example above, UIDs will be unique within a project, and most likely they are already universally unique. The likelihood of collisions (i.e. two UIDs from unrelated projects referring to different data fragments) decreases considerably when UIDs are only considered to be unique within a given source. TSSIDs enable this. By further nesting TSSIDs within projects as identified by their (Short)DOI, this combination creates an efficient decentralized method to create guaranteed universal unique references to qualitative data fragments. Because the first element allows locating the ROCK sources containing the data, and the second and third elements allows locating the data fragment within those files, this makes the data fragment instantly Findable (as in Findable, Accessible, Interoperable, and Reusable). Such identifiers (e.g. “p2bp_20230404T1600Z_7qzb6kw9”) lend themselves well for unequivocal reference in presentations or publications discussing a project, allowing researchers to look up exactly what is being discussed in its context.

Other special identifiers and recommendations

The ROCK 1.0 includes a number of additional special identifiers and recommendations. The class identifier “state” is reserved as the default identifier for states, enabling studying state transitions (see Zörgő et al., 2025). The code value “urcid” is reserved as a code value identifier that allows including a machine-readable link to a machine-readable codebook in every source. URCID stands for Unique ROCK Codebook Identifier, and it is either a URL that when visited serves the file with the codebook, or the relative path to a local file (in a project) containing the codebook. The ROCK codebook is either a spreadsheet or a YAML file in a specific structure (see section “Codebook specification” below). Another recommendation that is worth mentioning is to include a header with ROCK comments to provide readers with human-readable information about the data file and links to, for example, a project’s repository.

Because the ROCK was developed to be human- as well as machine-readable, ROCK files are plain text files. To code audio or video files, the recommendation is to use class instance identifiers to encode the time at specified time intervals. For example, the following brief ROCK source can indicate the codes applied to the first 10 seconds of a recording of a person performing a search task:

```
[[uid=80pgnsgk]] [[ts=00.00.00]]
  [[behavior>typing]]
  [[screen>google]]
[[uid=80pgnsgl]] [[ts=00.00.05]]
  [[google_search||how much fruit to eat per day healthy]]
  [[behavior>process_query]]
[[uid=80pgnsgm]] [[ts=00.00.10]]
```

This ROCK source can then be converted to a subtitle file and displayed as the video plays. For example, in the SubRip format (.srt), this would look like this:

```
1
00:00:00,000 --> 00:05:00,000
[[behavior>typing]]
[[screen>google]]

2
00:00:05,100 --> 00:10:00,000
[[google_search||how much fruit to eat per day healthy]]
[[behavior>process_query]]
```

This would show the first two codes as subtitles during the first five seconds, and the second two codes as subtitles during the next five seconds. Instead of the code identifiers, the code labels could also be shown (see section “Codebook specification” below).

Attributes

Attributes (variables holding additional information about class instances) can be specified either in separate YAML files (using the name “ROCK.attributes.yml” is recommended) or in YAML blocks in ROCK sources. YAML blocks in ROCK sources each begin and end with a line containing only three hyphens² (note that in separate YAML files, such delimiters should be omitted). Attributes have to be specified by *class instance*. This enables, for example, specifying demographic information about participants (e.g., what their age is) or characteristics of data collection locations (e.g., how busy it is). These attributes have to be stored in the root YAML node in a YAML document with key “ROCK.attributes”, the value of which should be a sequence (“list”) of maps (key-value pairs). Each map contains the attributes in key-value pairs, where the first key must be the *class identifier* (e.g., “caseId”) and the corresponding value the relevant class instance identifier (e.g., “p1” or “p2”; used to link the attributes to the relevant class instance). All other key-value pairs are imported as attributes, with the keys representing attribute names and the values the attribute values. Attributes enable, for example, querying coded data using attribute values. An example of attributes for two participants is shown below.

```
[[uid=7lm5s19c]] Here is some data.
[[uid=7lm5s19d]] Or *are* some data.

---
ROCK.attributes:

- caseId: "p1"
  age: "20-30"
  residence: "urban"

- caseId: "p2"
  age: "50-60"
  residence: "rural"

---

[[uid=7lm5s19f]] Here, potentially, data
[[uid=7lm5s19g]] could continue
```

Note how in this example, we somewhat defensively use quotation marks (“”) around strings, despite this not being necessary according to the YAML specification. This ensures that YAML parsers interpret the value as a string, which is a good precaution given some of YAML’s quirks (e.g. “no”, without quotes, sometimes being interpreted as “true”). Note also the added whitespace

² Note that in YAML, document streams can also be terminated with three dots. However, because this is much more likely to occur as a part of the data, this has been disallowed in the ROCK standard.

(newline and spaces) preceding the key-value combinations for each map; this facilitates visually distinguishing the blocks with attributes for each class instance identifier (e.g. each participant).

Aesthetics

When code structures are visualized, the nodes representing the codes and the edges representing the relationships between codes are endowed with aesthetic properties such as shapes, colors, and fonts. Aesthetics are specified in the *dot* language (Gansner and North, 2000) using *dot* node, *dot* edge, and *dot* graph attribute-value pairs. Like attributes, aesthetic configuration can be specified in separate YAML files (the recommended filename is “ROCK_aesthetics.yml”) or in YAML blocks in ROCK sources. Aesthetics have to be stored in YAML in the key “ROCK_aesthetics”, the value of which is a map with at most four keys: “nodes”, “edges”, “graph”, and “rank”. Both “nodes” and “edges” contain sequences (“lists”) of maps (key-value pairs), “graph” contains a single map that directly specifies the graph attributes, and “rank” contains keys “min”, “max”, and “same”.

Note that the elements making up a YAML fragment are also called nodes; to disambiguate, we consistently refer to these nodes as YAML nodes. The YAML nodes with the “nodes” and “edges” keys contain sequences of maps, where each map has to contain a “type” key. If the value of “type” is “default”, the specified aesthetics are applied to all nodes or edges. If the value of “type” is anything else, it is a regular expression to be matched against code identifiers or against edge types when network coding is used, where the aesthetics of the last encountered type that matches override earlier matches (allowing specifying “cascading” aesthetics). Examples of valid attribute-value pairs are “fontname: Arial”, “shape: box”, “color: black”, and “arrowhead: diamond” (note that YAML uses colons to separate keys and values, whereas *dot* uses equals signs). Aesthetics for nodes can also be specified in the codebook (see below).

The “graph” map contains graph attributes, such as “layout” (the layout engine to use; e.g. “dot”, “neato”, or “fdp”), “rankdir” (the rank direction; e.g. “LR” for “from left to right” or “TB” for “from top to bottom”), and “outputorder” (whether the draw “edgesfirst” or “nodesfirst”).

The “rank” can be used to organize the nodes. Each of “min”, “max”, and “same” contain a single character string or a sequence of character strings that are regular expressions that are matched against code identifiers to determine where to draw the corresponding nodes. Nodes that represent codes with code identifiers matching the regular expression at “min” are displayed at the first side specified in the rank direction (i.e. when “rankdir” is “LR”, at the left side); nodes that represent codes with code identifiers matching the regular expression at “max” are displayed at the last side specified in the rank direction (for “LR”, at the right); and for the sequence specified in “same”, nodes that represent codes with code identifiers matching each of these regular expressions are displayed at the same rank (in between the “min” and “max” nodes). This “rank” aesthetic is particularly helpful when using network coding to achieve a given visual organization of the resulting diagrams.

An example of an aesthetics specification is shown below. This example specifies two edge types (“causal” and “structural”) and one node type (“~bcp.”). These types are regular expressions matched against code identifiers. For nodes, that can be flat codes, tree codes, or network codes. Since only in networking coding, relations between codes are specified, regular expressions for edge types are only matched against relationship type codes (i.e. the third code identifier in a network coding pattern; see section “Codes”). The “nodes” YAML node also specifies the default layout for nodes that do not match any of the node type regular expressions. The “graph” YAML node specifies that the Dot engine should be used and that the graph should be laid out from left to right. Finally, the “rank” YAML node specifies that nodes representing codes with a code identifier that matches regular expression “bcp.” should be placed at the minimum rank (in this case, the left side).

```
---
ROCK_aesthetics:

  edges:

    - type: causal
      color: black
      style: solid

    - type: structural
      color: blue
      style: solid
      arrowhead: dot

  nodes:

    - type: default
      fillcolor: "#FFFFFF"
      fontcolor: "#000000"
      stroke: "#000000"
      shape: box
      style: rounded, filled

    - type: "~bcp."
      fillcolor: "#8ee9ff"
      fontcolor: "#000000"
      stroke: "#000000"
```

```
graph:
  layout: dot
  rankdir: LR
  outputorder: edgesfirst

rank:
  min: "bcp_"
  max: "tar_"
  same:
    - "apl_"
    - "det_"
---
```

Codebook specification

A *codebook* (either a predefined deductive codebook, or an inductively produced codebook) can be specified in three ways. First, like attributes and aesthetics, a codebook can be specified in a separate YAML file (the recommended name is “ROCK_codebook.yml”) or in a YAML block in a ROCK source. However, a codebook can also be specified in a spreadsheet file, the format of which will be explained later.

Codebook specification in YAML

In YAML, a ROCK codebook has to be stored in a YAML node with the key “ROCK_codebook” (or alternatively “ROCK_codes”, which is typically used for an incomplete codebook section). The “ROCK_codebook” YAML node contains a sequence named “codes”³, where each YAML node in the sequence represents one code. Each such YAML node contains a key “id” containing the code identifier (to enable machine-readability and interoperability); a key “label” containing the (human-readable) code label; a key “description” containing the code description; and a key “instruction” containing the instructions for applying a code. More advanced key-value pairs that can be specified for each code are “ucr” (containing the code’s Unique Code Reference, UCR; the code identifier appended with an underscore and a *SQUID* to produce a brief universally unique identifier for the code); “ucr_prefix” (a URL that can be prepended to the code’s UCR to produce a URL pointing to the code’s entry in a code repository; this code-level specification overrides a “ucr_prefix” specified in the codebook metadata); “ucr_url” (the full URL that points to the code’s entry in a code repository; this overrides all other specifications that allow derivation of that URL); and “ucid” (a URL containing a Unique Construct Identifier, UCID, that points to that construct’s entry in a construct repository, e.g. https://psycore.one/expAttitude_expectation_73dnt5z1).

In addition, a code’s YAML node can contain a sequence named “examples”, which contains a sequence of YAML maps. Each of these examples contains a key “example_fragment” containing the data fragment; a key “obviousness” containing either the text “core” or the text “edge”; a key “match” containing either the text “match” or “mismatch”; and a key “explanation” containing the explanation as to why this data fragment is an edge case or not, and why it is a match or a mismatch. A code’s YAML node can also contain one or more key-value pairs specifying *Dot* aesthetics (see section “Aesthetics”). Aesthetics specified in a code’s codebook entry take precedence over aesthetics specified in general aesthetics directives (in “ROCK_aesthetics” or in the codebook; also see section “Aesthetics”). A code’s YAML node can contain a sequence named “children”, which can contain other code’s YAML nodes. This nesting can have an arbitrary depth. Note that if a codebook specifies children for one or more codes (i.e. has nested codes), this requires tree coding as organizational mode.

Finally, a code’s YAML node can contain a sequence named “relationships”, which contains maps of two key-value pairs each: key “to_code_id” with a value that specifies the code identifier of the “receiving code”; and key “relationship_type” with a value that specifies the relationship type. Relationship types can be arbitrary when using network coding: for example “causal_influences_positive” or “structural_contains”. When using tree coding (also known as hierarchical coding), the relationship type can only be “parent_to_child” or “child_to_parent” (it is recommended to use the same throughout the codebook). When code YAML nodes contain a “children” YAML node, and so relationships are implicitly specified in the codebook structure, the “relationships” YAML node is ignored.

Additionally, the “ROCK_codebook” YAML node can contain a map named “metadata”, where each key-value pair specifies metadata about the project. The following metadata keys are defined: “title” (the project’s title); “authorString” (the codebook contributors in a human-readable format); “authorIds” (the codebook contributors in a machine-readable format, preferably ORCIDs); “date” (the date when the codebook was published); “organizational_mode” (the *organizational mode* of the codes in the codebook; must be “tree” or “hierarchical” if the codebook contains nested codes; “flat” if the codebook only contains unrelated codes; or “network” if the codebook contains network codes); “version” (the codebook’s version); “comments” (any comments about the codebook); “UCSID” (an optional Unique Codebook Specification Identifier, the URL to the codebook in a codebook repository); “UCR_prefix” (the prefix prepended to every code’s Unique Code Reference to produce the URL pointing to its entry in a code repository; note that this URL can also be directly specified in each code’s YAML node); “positionality” (the contributors’ positionality statements); “context” (any relevant descriptions of the project’s context); “narrative_process_documentation” (a URL pointing to the documentation describing the considerations, decisions, justifications, and other process characteristics that together with the context and positionality statements form the codebook’s provenance); “process_documentation_directives” (the directives forming a machine-readable version of the narrative process documentation); “UCSID_version” (the version of the Unique Codebook Specification that is used; this determines the codebook’s structure); and “URCID” (a URL to the codebook or a link to

³ Typically, there will only be one coding scheme, and as specified in section “Coding Schemes”, when no coding scheme is specified, the default coding scheme “codes” is implicitly used. When using multiple coding schemes in parallel, the name of each YAML node that holds the coding scheme’s codes in the sequence it stores is the coding scheme’s identifier.

a local file containing it; see section “Other special identifiers and recommendations”). Optionally, other key/value pairs can also be specified.

The “ROCK_codebook” YAML node can also contain a map named “aesthetics”, which is structured the same way as a “ROCK_aesthetics” YAML node (see section “Aesthetics”). Like aesthetics specified in a “ROCK_aesthetics” YAML node, aesthetics specified in a codebook’s “aesthetics” section specify general aesthetic directives that are used if no code-level aesthetics are specified in the codebook. Aesthetics specified in a “ROCK_aesthetics” YAML node also take precedence over those in a codebook’s “aesthetics” section, to enable application of varying looks to using the same codebook (aesthetics specified at the code level do take precedence over those specified generally).

An example of a very rudimentary codebook is shown below.

```
{--
ROCK_codebook:
  codes:

    - code_id: abuse
      code_label: Abuse (any type)
      code_description: >
        Explicit reference to or assumed instances of abuse.
      code_instruction: >
        Apply this code to a data fragment if it is clear that there
        is a reference to abuse, but it is not clear whether the
        abuse is emotional or physical. If that is clear, instead
        apply 'emotional' or 'physical'.
      fillcolor: "#FFCCCC"
      color: "#440000"
      code_children:

        - code_id: physical
          code_label: Physical abuse

        - code_id: emotional
          code_label: Emotional abuse

    - code_id: dissonance
      code_label: Dissonance

    - code_id: loss
      code_label: Loss
{--
```

The structure of a complete codebook is shown below. For the sake of space, this codebook contains only very minimal content. It contains two codes, with labels “Physical abuse” and “Abuse (any type)”. One example is provided of a data fragment that is an edge case that matches the “Physical abuse” code. A relationship of that code is specified, specifically that it is a child of the code with label “Abuse (any type)”. The only metadata field that is set is the project’s title, and then in addition to the code-level aesthetics specified for the “Physical abuse” code, some default aesthetics for nodes are specified.

```
{--
ROCK_codebook:
  codes:

    - code_id: A.physical
      code_label: Physical abuse
      code_description: >
        Explicit reference to or assumed instances of physical abuse.
      code_instruction: >
        Apply this code to a data fragment if [...]

      fillcolor: "#FFCCCC"
      color: "#440000"

      code_examples:

        - example_fragment: >
            You know, these streets will take something beautiful
            and they'll just like crush it and just like make
            sure that that's nothing (mumbles).
          obviousness: edge
```

```

    match: match
    explanation: >
      The explanation of why this fragment contains the
      characteristics that justify coding it with this code

  code_relationships:

    - to_code_id: abuse
      relationship_type: child.to.parent

  - code_id: abuse
    code_label: Abuse (any type)

metadata:
  - title: "Some project title"

aesthetics:

  nodes:

    - type: default
      fillcolor: "#DDDDFF"
      fontcolor: "#000044"
      stroke: "#000044"
      shape: box
      style: rounded, filled
{--

```

Codebook specification in a spreadsheet

ROCK codebooks can also be specified in a spreadsheet format, such as a LibreOffice OpenDocument Spreadsheet (a .ods file), an Office Open XML Workbook (an .xlsx file), or a Google Sheet. Such a spreadsheet workbook must have worksheets “metadata”, “codes”, “examples”, “aesthetics”, and “relationships”. Worksheet “metadata” must contain columns “field” and “contents”, where the “field” column contains the keys listed in the paragraph describing the contents of the “metadata” map in the “ROCK_codebook” YAML specification, and the “content” column contains the corresponding values. Worksheet “codes” must contain columns “code_label” (containing a code’s label), “code_description” (with the description of the code), “code_instruction” (the coding instruction for the code) and “code_id” (the code’s identifier). Optionally, it can contain columns “comments” (with comments), “ucr”, “ucr_prefix”, “ucr_url”, “ucid_url” (these four contain the same information as described in the description of each code’s YAML node in the “codes” sequence in the “ROCK_codebook” YAML node), as well as columns with names that are valid *dot* node attributes, such as “shape” and “fillcolor”. The “examples” worksheet must contain columns “code_id” (the identifier of the code to which the example in that row pertains), “example_fragment” (the data fragment), and “obviousness”, “match”, and “explanation” (again, see the description of the code YAML nodes in the “codes” sequence). The “aesthetics” worksheet must contain columns “target” (specifying whether a row sets a “graph”, a “node”, or an “edge” attribute); “type” (specifying either “default” or a regular expression against which codes are matched), “attribute” (the name of the Dot attribute), and “value” (the attribute value). The “relationships” worksheet must contain columns “from_code_id” (the identifier of the code from which the relationship starts), “to_code_id” (the identifier of the code from where the relationship ends), and “relationship_type” (the type of relationship; recall that for tree coding, this must be either “parent.to.child” or “child.to.parent”).

Coding Schemes

It is possible to specify that codes are a part of different parallel coding schemes. Coding scheme identifiers precede code identifiers, separated by two dashes. For example, to specify that a code with identifier ‘codeId’ is attached, within a coding scheme identified by ‘schemeId’, the following pattern is used: “[schemeId--codeId]”. Because using multiple parallel coding schemes is relatively rare, coding scheme identifiers are optional. If omitted, “codes” is used as a default coding scheme identifier. Code identifiers must be unique within their coding scheme; in this sense, coding schemes can be considered as namespaces.

ROCK projects

A ROCK project consists of a YAML file with the project’s metadata called “ROCKproject.yml”, optionally one or more ROCK sources, and optionally one or more additional files (such as “ROCK_attributes.yml”, “ROCK_aesthetics.yml”, and “ROCK_codebook.yml”). For convenience, these can be stored in a .zip archive with the extension “.ROCKproject”. ROCK projects can optionally contain additional YAML files with configurations for specific applications using the ROCK. For example, the {rock} R package (see section “Implementation of the ROCK”) will in a future version include a file “_rock-R-pkg.yml” with many of its default settings, so these can easily be changed at the project level. Similarly, the interface to the Reproducible Open Coding Kit (iROCK; <https://i.rock.science>) application presently under development includes a file “_iROCK.yml” that holds that application’s settings. These YAML files must be located in the project’s root directory.

The project’s “ROCKproject.yml” file must contain a single “ROCKproject” YAML node, which in turn contains a “project” YAML node, a “codebook” YAML node, a “sources” YAML node, and optionally a “workflow” YAML node. The “project” YAML

node is a map that contains key “title” (containing the project’s title); key “authors” (containing the author names as one text string); key “authorIds” (containing a sequence of maps, each map containing key-value pairs for “display_name”, “orcid”, and “shorcid”, as well as optional other key-value pairs), key “version” (containing the project’s version), key “ROCK_version” (containing the version of the ROCK used in the project), key “ROCK_project_version” (containing the version of the ROCK project specification, e.g. this YAML file, used in the project), key “date_created”, and key “date_modified” (these last two contain the date and time at which the project was created or modified, as ISO 8601 dates, preferably converted to the UTC timezone). The “_ROCKproject” YAML node can also contain other elements (none have been defined as yet, but it can be useful to be able to store additional information).

The “codebook” YAML node contains two keys: “urcid” and “embedded”. The “urcid” key stores the Unique ROCK Codebook Identifier, which is either a URL to a ROCK codebook in the format specified in the “Codebook specification” section or and the relative path to a local file with the codebook. The “embedded” key stores the codebook as specified in that “Codebook specification” section. Note that for the sake of accessibility and portability (i.e. use or adaption in different projects), ideally codebooks are not embedded in project files but stored in dedicated YAML files or spreadsheets (see section “Codebook specification”).

The “sources” YAML node describes the data sources in this project. It contains keys “extension” (containing the extension of files that are ROCK sources, normally “rock”), “regex” (an alternative way to specify which files are ROCK sources, containing a regular expression to match against filenames), “dirsToIncludeRegex” (containing a regular expression that matches directories in the project’s root that should be searched for ROCK sources), “recursive” (containing “true” if those directories should be searched recursively (i.e. also subdirectories), and “false” if they should not), “dirsToExcludeRegex” (a regular expression specifying directory names that should be excluded when recursing into the directories matching “dirsToIncludeRegex”), and “filesToIncludeRegex” and “filesToExcludeRegex”, which are matched against the final set of obtained files, and can be used for more fine-grained filtering. Many of these will be set to “~” (the tilde is the YAML symbol for “null” or “no content”).

The optional “workflow” YAML node contains two YAML nodes, “pipeline” and “actions”. These can be used to specify how the data flows through the project. The “pipeline” node contains a sequence of maps, where each map describes one stage in the pipeline with three keys: “stage” (the stage identifier), “dirName” (the directory holding ROCK sources in that stage), and one of “nextStage” or “nextStages” (one or more next stages that the data can progress to). A “nextStage” key contains a value that is the identifier of the next stage the data progresses to if processed in the current stage. A “nextStages” key contains a sequence of maps, where each map contains keys “nextStageId” (containing the identifier of the next stage) and “actionId” (the identifier of a YAML node in the “actions” YAML node: this is the action to apply to move the data to that next stage). The “actions” YAML node contains a sequence of maps with the keys “actionId” (containing the action’s identifier), “language” (containing the programming language in which the action’s script is specified), “dependencies” (a single value or sequence listing any packages to be loaded before running the action’s script), and “script” (the script to run when progressing from the current stage to the next stage). This script can use values from the pipeline YAML node. Values in the current stage can be accessed using “{currentStage::x}”, and values in the next stage can be accessed using “{nextStage::x}”, where “x” should be replaced with the key of the required value. For example, “{currentStage::dirName}” refers to the current stage’s directory, and “{nextStage::dirName}” to the next stage’s directory. The workflow functionality is used to support data management hygiene in projects.

Directives for software that works with ROCK files

The ROCK standard implies a number of directives that parametrize the importing and processing of ROCK projects and ROCK files. These are described here. Note that there are more ways than one to crack a nut, so these directives are opinionated and should be seen as recommendations rather than fully prescriptive.

Importing ROCK project files

Since project files are zip files (but with the extension “.ROCKproject”), a sensible first action upon importing or opening a project file is to extract the contained files to local device storage (e.g. a hard drive if that device is a personal computer). ROCK projects are not necessarily bundled into one such zip archive, in line with the ROCK’s emphasis on striving to be human-readable even without dedicated software. Therefore, ideally applications do not directly extract these files to an internal (e.g. temporary) directory, but ask the user where they would like the extracted version of the project to be stored (perhaps suggesting a sensible default, which may be user-configurable and may be stored in the software’s YAML settings file). Once the project has been imported and extracted to a local directory, it can be treated as a ROCK project as described in the following section.

Working with ROCK projects

When processing a ROCK project, an application should respect the settings in the project’s “_ROCKproject.yml” YAML file. This means, for example, that the directives specified there determine which files are considered ROCK sources with data. Similarly, information about the codebook and aesthetics specified there should be used in data visualisations. Default filenames that should also be searched and read if present are “ROCK_attributes.yml” (containing the attributes for each class instance), “ROCK_aesthetics.yml” (containing the aesthetics), and “ROCK_codebook.yml” (containing the codebook). Directives specified in these separate files override information in the project file. This enables quickly swapping, for example, one set of aesthetics for another (allowing efficient “theming” of visualisations). However, by default, aesthetics specified at the code level in a codebook should take precedence over generally specified aesthetic directives. Of course, ideally applications allow users to control these precedence rules through application-level or project-level settings (note that applications can store their own settings in ROCK projects).

Parsing ROCK sources

When parsing a ROCK source, we recommend the following order of extracting ROCK elements: First, all lines starting with a hash symbol (“#”) should be ignored. Second, any embedded YAML fragments should be extracted and processed. Third, the position of all structuring information (section breaks, anchors, nesting) should be extracted. Fourth, notes should be extracted. Then, utterance identifiers should be extracted (if these are specified). Finally, what remains are the class instance identifiers and ‘regular’ ROCK coding patterns, which can then be processed in any order. These codes, as well as notes, are attached to data

fragment on the line on which they occur, unless that line contains no data, in which case they are attached to the closest preceding line with data. In principle, then, after parsing, a ROCK source reduces to the contained data accompanied by the information encoded conform the ROCK, attached to the data fragments.

Creating ROCK sources

New ROCK sources will typically be created by importing data from a plain text file, a word processor file (e.g. in OpenDocument format (.odt), Open Office XML format (.docx), or a Portable Document Format (.pdf)), by copy-pasting such data, or by typing in the data directly. Given the ROCK line-based nature, one of the first things that applications that work with ROCK sources do is support users in cleaning the data and prepending utterance identifiers to every line. The data cleaning typically consists of applying whichever definition of an utterance is chosen by the user. Since the ROCK delimits utterances using line breaks, this boils down to, for example, word wrapping the source at a given number of characters (e.g., 40), or inserting a line break after every sentence. In addition, the application ideally produces a series of SQUIDs to use as utterance identifiers.

Producing SQUIDs

Short Quasi-Unique Identifiers (SQUIDs) can be produced as follows. First, take the local time, and if that is not supplied as a Unix timestamp, convert it to that format. Then, multiply it by 100 and round it, so that the number of centiseconds that passed since the first of January 1970 at 00:00 UTC is obtained. Finally, convert this number to a base thirty decimal system, consisting of the 10 Arabic digits (0-9) followed by the Latin alphabet in lowercase and in alphabetic order but omitting the vowels (a, e, i, o, and u) as well as the v (so, “b” represents 10; “g” represents 14; “s” represents 24; and “z” represents 29; “3b” represents 100, so one second; “6n0” represents 6000, so one minute; and “fb00” represents 360000, so one hour). In R, the package {squids} can be used for this [Peters (2025); also see <https://cran.r-project.org/web/packages/squids/vignettes/squids.html>].

The ROCK glossary

This is a glossary of ROCK terms and their definitions as well as related concepts and a ROCK example.

Term	Definition	Examples in the ROCK standard
Anchor	A sequence of characters that encodes a specific point in the data collection.	--+-{ 2022-10-17 10:07:32.752 CEST (UTC+2) }-+--
Attribute	When collecting multiple data streams (e.g. transcribed audio and video) these can be used to synchronize codings from these data streams and project all codings onto the primary data stream.	
Class	Data about a <i>class instance</i> . These are specified in YAML fragments in ROCK sources. Attributes are variables (i.e. things that can have two or more possible values), for example “age”, “schoolType”, or “residenceType”.	age: "20-30"
Class identifier	A set of similar data providers (such as participants) or contextual elements relating to the data collection, processing, or analysis. Examples of classes are participants, interview locations, times, or coders.	
Class instance	The unique identifier for a class. This identifier is added to ROCK sources, together with class instance identifiers, to specify that the following data fragments are associated to an instance of that class. Examples are “participantId”, “locationId”, “timeId”, or “coderId”.	
Class instance identifier	A class member, such as a single participant (“Alex”), interview location (“the library”), time (“morning”) or coder (“NAME”).	
	The identifier that identifies a class instance. This identifier is added to ROCK sources to specify that the following data fragments are associated to that class instance. Examples of class instance identifiers are “Alex”, “theLibrary” or “the.library”, “morning”, and “SHORCID”.	[[participantId:007]]

Term	Definition	Examples in the ROCK standard
Code	A representation of a pattern with a specified set of characteristics. A code can represent, for example, “advantages of staycations”, “perceived social norms”, or “power differential”.	
Code description	Where <i>code labels</i> are very short descriptions of a code, code descriptions are longer, providing a clear definition of the pattern or phenomenon represented by the code.	
Code identifier	Code identifiers are identifiers for codes. Examples are “adv_staycation”, “percSocNorms”, and “powDiff”.	[[adv_staycation]]
Code instruction	The instruction for a coder as to when to apply the corresponding code. Where the <i>code description</i> describes what the code represents, the code instructions is less of a definition and more operational instruction describing which characteristics of the data the coder should look for that qualify an utterance for application of the corresponding code.	
Code label	A human-readable label for a code. These are normally short descriptions of the phenomenon the code is meant to represent. As such, examples are “advantages of staycations”, “perceived social norms”, or “power differential”.	
Code value	A <i>coding pattern</i> allowing specification of a value for a code identifier. This can be used, for example, for magnitude coding, or for collecting participants’ ages or other information during interviews.	[[participantAge 34]]
Codebook	A structured collection of data describing one or more codes, including for each code their <i>code identifier</i> , <i>code label</i> , <i>code description</i> , <i>code instruction</i> , relevant examples, and relevant aesthetics, as well as relationships between codes and optionally metadata.	
Coding	A <i>coding pattern</i> .	
Coding instance	A <i>coding pattern</i> .	
Coding pattern	One or more code identifiers as (potentially) applied to (parts of) one or more <i>utterances</i> .	[[parentCode>childCode]]
Comment	A ROCK comment is a line that is ignored by ROCK parsers. ROCK comments starts with a hash or pound sign (#).	### This is a comment and should be ignored by parsers
Data fragment	An <i>utterance</i> .	
Dot	A language enabling specification of diagrams, that can then programmatically be visualized using the GraphViz library.	
Flat coding	An <i>organization mode</i> where codes have no relationship to each other.	
Hierarchical coding	See <i>tree coding</i> .	
Identifier	A sequence of characters that is unique within a project and that starts with a lower or upper case Latin letter (a-z, A-Z), followed by zero or more lower or upper case Latin letters, zero or more Arabic digits (0-9), and zero or more underscores (.).	idCombining_camelCase_and_snake_case

Term	Definition	Examples in the ROCK standard
Nesting	Nesting can mean two things. First, utterances can be nested, for example when they are obtained from social media such as Mastodon. Nested utterances have exactly one “parent” utterance (typically, the utterance they are a response to). Second, codes can be nested when using tree coding as the <i>organizational structure</i> for codes.	<code>~ A nested utterance</code>
Network coding	An <i>organizational mode</i> where codes can have arbitrary relationships to each other.	<code>[[fromCode->toCode relationshipType]]</code>
Note	A note is a text attached to a data fragment, optionally accompanied by one or more keyed values.	<code>--{ This is a brief note attached to it. }--</code>
Organizational mode	How codes are organized relative to each other. With flat coding, it is not possible to express relationships between codes. With tree coding, codes have a hierarchical (parent-child) relationship (i.e., are nested). With network coding, codes can have arbitrary relationships.	
Section break	The coding used to represent segmentation for a given segmentation type. Each segmentation type has a section break identifier.	<code>{--<<turn_of_talk>>---}</code>
Section break identifier	The <i>identifier</i> representing the type of segmentation applied in a section break.	<code>turn_of_talk</code>
Segmentation	The deliberate splitting of a data stream into segments. Segmentation can be used, for example, to clearly distinguish data representing answers to specific questions, turns-of-talk in a focus group, or shifts in a topic. <i>Utterances</i> are also segments (typically the smallest segment occurring in a <i>source</i>).	
SQUID	A Short Quasi-Unique Identifier; an 8-character text string that is almost always universally unique (Peters, 2025).	<code>71m5s18f</code>
Source	A plain text file with data (or a representation of the data) that can be coded (i.e., that <i>coding patterns</i> can be added to).	
Time Stamped Source Identifier	A unique identifier for a source that consists of the time the data collection of the data in that source started converted to the UTC time zone and omitting seconds, and represented in the ISO 8601 standard.	
Tree coding	An <i>organizational mode</i> where codes are nested within each other (e.g. have a hierarchical relationship).	<code>[[parentCode>childCode]]</code>
TSSID	A <i>Time Stamped Source Identifier</i> .	<code>20250809T0900Z</code>
UID	An <i>utterance identifier</i> .	<code>71m5s18f</code>
Utterance	A data fragment with a uniform definition throughout a source, such as “a sentence”, “40 characters”, or “data registered over the course of 10 seconds”. In the ROCK standard, utterances are delimited by line breaks (ASCII character 10, unicode U+000A). Codes are typically applied at the utterance level, unless character-level coding is used.	
Utterance identifier	A Short Quasi-Unique Identifier (SQUID) that uniquely identifies an <i>utterance</i> .	<code>[[uid=71m5s18f]]</code>

Term	Definition	Examples in the ROCK standard
YAML	YAML is a recursive acronym that stands for “YAML Ain’t Markup Language”. It is a data serialization language designed to be simultaneously machine-readable and human-readable. YAML is used in the ROCK standard to specify structured data, such as attributes, aesthetics, and codebooks. YAML allows structuring data in nodes, where nodes can be named with keys, and each node contains either a sequence (an ordered list of nodes), a map (an unordered list of key-value pairs), or a scalar (a single value).	

REFI terms and ROCK terms

REFI established a set of terms and listed their synonyms in seven applications (Rotterdam Exchange Format Initiative, 2019). For readers familiar with this list, the relevant synonyms in the ROCK will be listed here. REFI and the ROCK use the same terms for Code, Codebook, and Coding. In the ROCK, a REFI Case would be a *class instance* (with “Cases” being the *class*, and something like “*caseId*” or “*cid*” being used as class identifier). A REFI Description can be multiple things in the ROCK: a *comment*, a *code description*, or a *note*. A REFI Graph comes closest to the diagram resulting when using *network coding*, and a Link is either a parent-child relationship between two codes when using hierarchical coding, or an edge between two codes when using network coding. A Note is a *note* or a *comment* in the ROCK, a Project is a *ROCK project*, and a Selection is either an ROCK *utterance*, when using utterance-level coding, or, when using character-level coding, a sequence of one or more characters distributed over one or more utterances. A REFI Set does not have an equivalent in the ROCK v1.0, but can be represented as a combination of one or more ROCK *attribute* values. A REFI Source is a ROCK *source*, and a REFI Transcript is also a *source*. A REFI User is a ROCK *user*, and a REFI Variable is an ROCK *attribute*.

The ROCK in the academic landscape

The ROCK was developed to facilitate application of open science principles in qualitative research, to catalyse innovation in qualitative research, and to progress towards machine-readability and interoperability of (coded) qualitative data, to ultimately enable Findable, Accessible, Interoperable, and Reusable qualitative data. In this section, we will briefly describe how the ROCK can be used by qualitative researchers, by developers, and how it facilitates Open Science.

The ROCK for qualitative researchers, methodologists, and developers

By using the ROCK for their projects, qualitative researchers avoid becoming locked in to a given software package. Many qualitative research applications are proprietary (i.e. incompatible with Open Science and the mandated transparency intrinsic to many scientific integrity codes of conduct), and using such software entails the risk of no longer being able to access the project at a later stage, for example because licenses expire or a researcher changes institution.

Second, individual researchers benefit from the ROCK’s facilitation of easy reuse of data across projects. The ROCK lends itself well to conducting cumulative projects, for example with students contributing over time. In addition to facilitating such temporally (or geographically) distributed data collection, the ROCK is also easy to implement in workflows with distributed coding. The attachment of UIDs to data fragments, the self-contained nature of ROCK sources, and the ease with which bespoke applications can be created, open up a wealth of opportunities. For example, envision a procedure where citizens contribute to data collection to a central hub. There, the data are checked for anonymization, cleaned, and utterance identifiers are attached. The resulting ROCK sources are then distributed to a number of independent coders. These attach codes, and then resubmit the coded sources. These can then be combined, the convergences and divergences in coding patterns can be analysed, and a reconciliation process can be implemented. When the reconciliation process is properly documented, this creates an entirely transparent process. Another example consists of groups of researchers at several institutions who pool collected data, perhaps even using a process to independently code each other’s data and periodically meet to discuss divergences. If such groups of researchers would also use Decentralized Construct Taxonomies to define the constructs they study (Peters and Crutzen, 2024), their data would become interoperable, enabling automatic pooling of data (note that the ROCK codebook standard already enabled specifying the Unique Construct Identifier or UCID for the construct or concept that a code aims to capture). This would enable studying where given contextual conditions are important and where they matter less or not at all.

Third, because the ROCK was designed as a relatively easy standard, its nature facilitates researchers with minimal coding skills (or researchers who collaborate with research software engineers) when they want to automate part of their coding process or their analysis of coded sources. This means that in qualitative work, the functionality implemented in existing software packages ceases to be a straitjacket: in principle, the ROCK opens coded qualitative data up for processing in any scriptable system. In additions, some existing implementations of the ROCK standard (see that section below), such as Shiny ROCK Emerald (<https://shiny.rock.science/emerald>) and iROCK (<https://i.rock.science>) already parse one or more ROCK sources into a so-called Qualitative Data Table: a tabular representation of the coded data, where rows correspond to utterances, and 0/1-coded columns indicate which codes were applied to each utterance. This means that even researchers with no coding skills but familiarity with spreadsheet software (such as LibreOffice Calc) can enjoy this freedom. This also lowers the threshold for methodological innovation (see the penultimate paragraph in this section).

For developers, the ROCK's plain-text foundation makes it relatively easy to write packages that process ROCK files. This means that for developers of existing software that works with qualitative data, adding support for the ROCK standard is relatively easy. It is also relatively easy to write scripts to work with ROCK files, for example to automate coding based on regular expressions, or to create new data visualizations. Finally, combined with other tools such as construct repositories [for more details, see Peters and Crutzen (2024); and for an example, see <https://psycore.one>], the ROCK enables interoperability of coded qualitative data. As these standards evolve, it will become possible to acquire relevant metadata and support researchers in their coding or analyses.

Finally, for the scientific community, the ROCK opens the door to methodological innovation through two routes. First, the ease of working with ROCK files means it is relatively easy (and therefore, cheap) to realize the functionality that implements novel ideas. Second, because basic coding skills suffice to produce scripts that operate on ROCK files, even academics that do not code but can script can themselves implement innovative ideas. Since methodological innovations are typically invented by academics to address epistemological needs or opportunities, they often occur during substantive work, and the ROCK lends itself well to this way of working. This means that the ROCK brings methodological innovation in qualitative work within reach of all academics, holding the promise of more diverse and prolific progress. One of the earliest examples is Qualitative/Unified Exploration of State Transitions [QUEST; Zörgő et al. (2025)], where class instance identifiers were used to code states, which were then processed to visualize how participants proceeded through these states.

In addition to enabling methodological innovation, the ROCK also allows varying epistemic affordances. In scientific endeavours, the ontological and epistemological foundations and theoretical context together manifest as a methodological spectrum; but this spectrum itself manifests in a final operational layer, often implemented in software applications. The design of this operational layer can then allow researchers to leverage certain methodological opportunities, but can also block a given methodological approach, or by its defaults nudge researchers to make certain methodological decisions. Two of the most well-known examples of the latter both concern SPSS. First, the protracted absence of Cohen's d from SPSS⁴ has virtually waylaid wide adoption of effect size reporting and systematic a priori sample size calculations for decades. Second, the default method for dimension reduction (principal components analysis) is almost never appropriate for the type of data typically collected in the social sciences. A third example concerns qualitative research, where typically, coding occurs either flatly (without any relationship between codes) or in a tree structure (with hierarchical parent-child relationships between codes), with the ROCK enabling coding in a network [by specifying three codes: from, to, and relationship type; Peters et al. (2022a)]. The chosen organizational mode represents the ontological position that it can adequately (and ideally, accurately) represent the phenomena of interest. Most qualitative programs not only nudge researchers towards tree coding but offer it as only organizational mode, effectively amounting to epistemic blinders. The ROCK lends itself well to a proliferation of interfaces for coding, allowing exploration of the epistemological implications of the epistemic affordances (or blinders) implemented in each.

The ROCK and Open Science

Qualitative researchers have for a long time been aware of the ways their positionality may shape their analyses, interpretations, results, and conclusions. In quantitative research, awareness of the potential biasing⁵ influences of undisclosed researcher degrees of freedom (e.g., Simmons et al., 2011; Gelman and Loken, 2014) has recently been made salient by projects such as the Many Analysts project (Silberzahn et al., 2018). In this light, it is somewhat surprising that between roughly 2010 and 2025, the practices around sharing insight into the data and analysis process seem to have improved faster for quantitative methods than for qualitative methods.

The ubiquitous nature of quantitative data in combination with the proprietary nature of the most popular software packages for qualitative research may play a role here. While the comma-separated values file format was already in use since 1972, the Rotterdam Exchange Format Initiative was only founded in September 2016, and the ROCK only started to take shape in December 2018. As such, a delay in uptake can be expected. The higher accessibility of the ROCK standard compared to the REFI-QDA file format may contribute to faster dissemination of this open standard.

Another aspect may be the rigor and level of detail with which quantitative analyses can be specified in a number of open formats. The most used quantitative data analysis applications such as the proprietary SPSS, SAS and STATA, as well as the free and open jamovi (Navarro and Foxcroft, 2025; ŞAHİN and AYBEK, 2020), JASP (Love et al., 2019), R (R Core Team, 2025) and Python (Van Rossum and Drake, 2009), all support exhaustive documentation of the analyses, either in plain text files or embedded in the project file. This makes it possible to comprehensively document the analysis process and the stages through which the data proceed, which in turn allows extensive justification of the decisions and provides important context, as well as facilitating assessment of risk of bias in evidence syntheses. For qualitative research, the analysis process is specified at best in extensive narrative documentation, which by virtue of their lack of structure and machine-readability can be stored in any format.

The ROCK format has the potential to change the playing field in two ways. First, its implementation in the `{rock}` R package (see next section) includes functions to, for example, add or remove codes, merge or split codes, and move codes (in a code structure in the tree coding organizational mode). By accompanying calls to these functions with comments, as is common for quantitative analyses, similar rigor and detail can be achieved; and the `{rock}` package can even take this one step further by supporting the specification of machine-readable justifications through an interface with the `{justifier}` R package (Peters, 2019). Second, because a set of ROCK sources are plain-text files, they afford the storage of intermediate versions of the data. For example, cleaned sources can be stored in a different directory or with different filenames from raw sources; the first coding sweep can be stored in yet another directory or with another filename; and so on. This facilitates inspection of the consequences of the decisions taken during the analytical process, much like this level of transparency is already achieved in most software for quantitative analysis.

⁴ Cohen's d was added in version 27, released in 2020; Cohen's seminal book was published in 1988; SPSS 1.0 was released in 1968, with version 2.0 released in 1983 and version 5.0 released in 1993.

⁵ We define 'biasing' here as undesirable and often unknown distortions of the results.

Implementations of the ROCK

At the moment, the ROCK has been implemented in a number of open source applications. The first implementation was in the `{rock}` R package [Peters and Zörgő (2019)]; the git repository is located at <https://codeberg.org/R-packages/rock>, the first version of which was published on the Comprehensive R Archived Network (CRAN) on the 14th of June 2019. In addition, a rudimentary HTML/CSS/JS application was developed, as well as a newer Electron application, both geared towards coding. These are available from <https://i.rock.science> (with source code available at <https://codeberg.org/sci-ops/i.rock.science-website> in the “original” subdirectory for the original version and at <https://codeberg.org/sci-ops/desktop-shiny-electron> for the Electron application).

In addition, a series of rudimentary Shiny apps have been developed. These Shiny ROCKs were each designed to fulfil one ROCK-related function. Shiny ROCK Amethyst was developed for cognitive interviews. It allows the user to upload one or more ROCK sources, parses them into a qualitative data table, and produces a Cognitive Interview heatmap. This heatmap shows which codes occur for which items. It has been superseded by Shiny ROCK Beryl, which has the same functionality. Shiny ROCK Crystal was developed for easy experimentation with the Qualitative Network Approach. It allows the user to paste a ROCK source and will then parse it and produce a network from the codes (also see Peters et al., 2022a). Shiny ROCK Diamond was developed as a very rudimentary coding interface that is optimized for use with touchscreens. Shiny ROCK Emerald was created to add UIDs to every data fragment (line) of a source. Shiny ROCK Feldspar was created to produce a Qualitative Data Table from `.rock` (or `.txt`) files in a `.zip` archive. The source code of these Shiny apps is available from <https://codeberg.org/sci-ops>, and the live apps can be accessed through <https://shiny.rock.science>. In addition, a more comprehensive Shiny app is under development as the new iROCK version (its git repository is at <https://codeberg.org/sci-ops/iROCK>).

Furthermore, `{qualchemy}` is under development, an R package developed to import, convert, and export to multiple file formats for qualitative projects (its git repository is located at <https://codeberg.org/R-packages/qualchemy>). Its first version will support the REFI-QDA standard as well as the ROCK standard, but it is designed to also allow easy extension to import and export to and from other open standards. Finally, an increasing number of projects use the ROCK. A number of examples are linked to from <https://rock.science>.

Discussion

We have introduced the first version of the Reproducible Open Coding Kit, a human- and machine-readable standard for (coded) qualitative data, and positioned it in the wider epistemological and methodological landscape of qualitative research. In this discussion, we will briefly touch upon a number of limitations, strengths, and future directions.

Limitations

The main strength of the ROCK is also its main weakness. By prioritizing human-readability as well as machine-readability, the representational power of the ROCK is relatively low. Given that ultimately, everything can be derived from the patterns in the applied codes (such as visualizations of the code structure, visualizations of code (co-)occurrences, and collection of data fragments coded with a given pattern of codes), this is partly a matter of lower convenience. However, it also means that the ROCK does not lend itself well to direct coding of non-textual data, such as audio, video, or images. As discussed in section “Other special identifiers and recommendations”, there are workarounds, but these are less user friendly than directly attaching codes to the data in its original medium.

Strengths

The main weakness of the ROCK is also its main strength. By prioritizing human-readability as well as machine-readability, the data that have been coded with the ROCK are optimally accessible, even to researchers, policymakers, or citizens without any dedicated software other than a plain text editor. The related simplicity of the standard further augments this accessibility to developers: creating scripts, packages or applications that code, process, or analyze ROCK sources is relatively easy. This is further enhanced by the use of YAML for data serialization of ROCK-related (meta)data, as YAML, too, is simultaneously human- and machine-readable.

In addition, the ROCK standard was designed to facilitate interoperability. As an open standard, it intrinsically realizes interoperability on the superficial level. However, to accomplish data that is Findable, Accessible, Interoperable, and Reusable to the fullest extent, the interoperability needs to extend to the conceptual level. That enables interoperability on data from different projects; for example, synthesizing data from various projects. The ROCK codebook standard provides for this; it enables specifying Unique Construct Identifier URLs for a code, allowing machine-readable specification of what the code represents [for more details, see Peters and Crutzen (2024); and for an example construct repository, see <https://psycore.one>]. Combined with the other features of the ROCK standard, this positions the ROCK well for use in distributed projects involving diverse groups. By grounding interoperability on the conceptual level in extensive descriptions and coding instructions for every code, each linked to comprehensive construct definitions, methodological rigor and epistemological accountability can be retained even at scale.

Conclusion

The Reproducible Open Coding Kit is an open standard for (coded) qualitative data that is accessible without dedicated software, relatively easy to implement in existing software, and designed for interoperability. It facilitates research workflows that are entirely in line with open science principles and lends itself well to conducting qualitative research at scale. Finally, through its open and accessible nature it furnishes many researchers with the means for methodological innovation of qualitative research.

Acknowledgements

This article has been funded in part by NWO Open Science grant, number OSF23.2.090.

References

- Ben-Kiki, O., Evans, C., and Ingerson, B. (2009). Yaml ain't markup language (yaml™) version 1.1. *Working Draft 2008*, 5(11).
- Bilder, G., Lin, J., and Neylon, C. (2022). The Principles of Open Scholarly Infrastructure.
- Brailas, A., Tragou, E., and Papachristopoulos, K. (2023). Introduction to Qualitative Data Analysis and Coding with QualCoder. *American Journal of Qualitative Research*, 7(3):19–31. <https://doi.org/10.29333/ajqr/13230>.
- Evers, J. (2018). Current Issues in Qualitative Data Analysis Software (QDAS): A User and Developer Perspective. *The Qualitative Report*. <https://doi.org/10.46743/2160-3715/2018.3205>.
- Evers, J., Caprioli, M. U., Nost, S., and Wiedemann, G. (2020). What is the REFI-QDA Standard: Experimenting With the Transfer of Analyzed Research Projects Between QDA Software. *Forum Qualitative Sozialforschung / Forum: Qualitative Social Research*, Vol 21:No 2 (2020): Challenging Times– Qualitative Methods and Methodological Approaches to Research on Time. Publisher: Forum Qualitative Sozialforschung / Forum: Qualitative Social Research.
- Gansner, E. R. and North, S. C. (2000). An open graph visualization system and its applications to software engineering. *Software: Practice and Experience*, 30(11):1203–1233. eprint: <https://onlinelibrary.wiley.com/doi/pdf/10.1002/1097-024X%28200009%2930%3A11%3C1203%3A%3AAID-SPE338%3E3.0.CO%3B2-N>.
- Gelman, A. and Loken, E. (2014). The garden of forking paths: Why multiple comparisons can be a problem, even when there is no “fishing expedition” or “p-hacking” and the research hypothesis was posited ahead of time. *Psychological bulletin*, 140(5):1272–1280.
- Gruber, J. and Swartz, A. (2004). Markdown.
- Love, J., Selker, R., Marsman, M., Jamil, T., Dropmann, D., Verhagen, J., Ly, A., Gronau, Q. F., Smíra, M., Epskamp, S., Matzke, D., Wild, A., Knight, P., Rouder, J. N., Morey, R. D., and Wagenmakers, E.-J. (2019). *j*bjjaspi/*bj*: Graphical statistical software for common statistical designs. *Journal of Statistical Software*, 88(2). <https://doi.org/10.18637/jss.v088.i02>.
- Navarro, D. and Foxcroft, D. (2025). *Learning Statistics with jamovi: A Tutorial for Beginners in Statistical Analysis*. Open Book Publishers, Cambridge, UK, 1 edition. <https://doi.org/10.11647/OBP.0333>.
- Peters, G.-J. (2019). justifier: Human and machine-readable justifications and justified decisions based on ‘yaml’. <https://doi.org/10.32614/CRAN.PACKAGE.JUSTIFIER>.
- Peters, G.-J. (2025). squids: Short quasi-unique identifiers (squids). <https://doi.org/10.32614/CRAN.PACKAGE.SQUIDS>.
- Peters, G.-J. and Crutzen, R. (2024). Knowing what we're talking about: Facilitating decentralized, unequivocal publication of and reference to psychological construct definitions and instructions. *Meta-Psychology*, 8. <https://doi.org/10.15626/MP.2022.3638>.
- Peters, G.-J. and Zörgő, S. (2019). rock: Reproducible open coding kit. <https://doi.org/10.32614/CRAN.PACKAGE.ROCK>.
- Peters, G. J., Zörgő, S., and van der Maas, H. L. J. (2022a). The qualitative network approach (qna). <https://doi.org/10.31234/OSF.IO/CVF52>.
- Peters, G.-J. Y., Zörgő, S., and van der Maas, H. (2022b). The Qualitative Network Approach (QNA). preprint, PsyArXiv. <https://doi.org/10.31234/osf.io/cvf52>.
- R Core Team (2025). *R: A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing, Vienna, Austria.
- Rotterdam Exchange Format Initiative (2019). REFI-QDA Project.
- Silberzahn, R., Uhlmann, E. L., Martin, D. P., Anselmi, P., Aust, F., Awtrey, E., Bahnik, S., Bai, F., Bannard, C., Bonnier, E., Carlsson, R., Cheung, F., Christensen, G., Clay, R., Craig, M. A., Dalla Rosa, A., Dam, L., Evans, M. H., Flores Cervantes, I., Fong, N., Gamez-Djokic, M., Glenz, A., Gordon-McKeon, S., Heaton, T. J., Hederos, K., Heene, M., Hofelich Mohr, A. J., Högden, F., Hui, K., Johannesson, M., Kalodimos, J., Kaszubowski, E., Kennedy, D. M., Lei, R., Lindsay, T. A., Liverani, S., Madan, C. R., Molden, D., Molleman, E., Morey, R. D., Mulder, L. B., Nijstad, B. R., Pope, N. G., Pope, B., Prenoveau, J. M., Rink, F., Robusto, E., Roderique, H., Sandberg, A., Schlüter, E., Schönbrodt, F. D., Sherman, M. F., Sommer, S. A., Sotak, K., Spain, S., Spörlein, C., Stafford, T., Stefanutti, L., Tauber, S., Ullrich, J., Vianello, M., Wagenmakers, E.-J., Witkowiak, M., Yoon, S., and Nosek, B. A. (2018). Many Analysts, One Data Set: Making Transparent How Variations in Analytic Choices Affect Results. *Advances in Methods and Practices in Psychological Science*, 1(3):337–356. <https://doi.org/10.1177/2515245917747646>.
- Simmons, J. P., Nelson, L. D., and Simonsohn, U. (2011). False-positive psychology: undisclosed flexibility in data collection and analysis allows presenting anything as significant. *Psychological science*, 22(11):1359–66. <https://doi.org/10.1177/0956797611417632>.
- UNESCO (2021). UNESCO Recommendation on Open Science. Programme and meeting document SC-PCB-SPP/2021/OS/UROS. Technical report, UNESCO.
- Van Rossum, G. and Drake, F. L. (2009). *Python 3 Reference Manual*. CreateSpace, Scotts Valley, CA.
- Zörgő, S., Peters, G.-J., Jeney, A., Kovács, S. D., and Crutzen, R. (2025). A multimodal analysis of online information foraging in health-related topics based on stimulus-engagement alignment: Observational feasibility study. *Journal of Medical Internet Research*, 27:e64901–e64901. <https://doi.org/10.2196/64901>.
- ŞAHİN, M. and AYBEK, E. (2020). Jamovi: An easy to use statistical software for the social scientists. *International Journal of Assessment Tools in Education*, 6(4):670–692. <https://doi.org/10.21449/ijate.661803>.